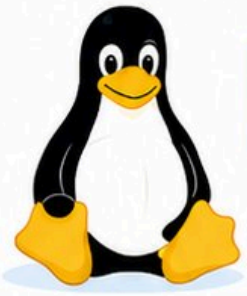


Linux Kernel Guide

What the kernel is, why it matters, and how it manages your computer.

The Linux kernel is the core of the operating system. It sits between applications and hardware, acting like a manager, translator, and traffic controller.

1. APPLICATIONS

Browser | Terminal | Database

2. LINUX KERNEL

Manages resources, provides services, and protects the system.

3. HARDWARE

CPU | RAM | Disk | Network

Why do we need a kernel?

Resource sharing
Lets many programs use CPU, memory, and devices safely and fairly.

Protection
Keeps programs from interfering with each other or the system.

Hardware control
Talks to devices and makes sure everything works correctly.

Abstraction
Hides hardware complexity and provides simple, consistent interfaces.

The kernel is like an airport control tower

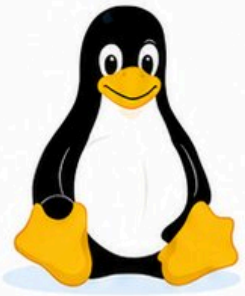
Airplanes (Programs)	Runways (CPU)	Gates (Memory)	Equipment (Devices)
request to land or take off.	decide who goes next.	hold planes while they wait.	do the physical work.

What you'll learn

- CPU scheduling
- Memory management
- Device drivers
- System calls
- Filesystems

Keep it simple: applications request services, the kernel manages the work, and hardware does the physical job.

Created By: Pankaj



User Space, Kernel Space, and System Calls

How applications safely ask the kernel for help.

User Space

Limited privileges

- Browser
- Python
- Bash
- Database

Kernel Space

Full privileges (trusted)

- Scheduler
- Memory Manager
- Drivers
- Filesystems

Applications run in user space with limited privileges. The kernel runs in kernel space and can directly manage hardware and core resources.

What is a system call?

Application
Your program

↓

C library / API
Convenient functions

↓

System call boundary
Switch to kernel mode

↓

Linux Kernel
Performs the requested work

↓

Device or file
Hardware or storage accessed

↓

Result returned
Back to your application

Common system calls

`open()`	Open a file or device
`read()`	Read data
`write()`	Write data
`fork()`	Create a new process
`execve()`	Replace process image
`mmap()`	Map memory
`socket()`	Create a network socket

Think of a system call as a request form to a secure control room.

Keep it simple: Programs cannot directly control hardware. They request approved services through system calls.

CPU Scheduling: Who Runs Next?

How the kernel shares CPU time across many tasks.

A CPU core can run only one instruction stream at a time, but many programs want attention. The scheduler decides who runs next.

1. PROCESS VS THREAD

Process

Running program with its own memory and resources.

Thread

Execution path inside a process.

One process with multiple threads

2. HOW SCHEDULING WORKS

Runnable tasks

- Browser
- Terminal
- Music
- Backup Job

Linux Scheduler
Decides who runs next.

CPU cores

- CPU Core 0
- CPU Core 1

4. GOALS OF SCHEDULING

Fairness
Give each task a fair share.

Responsiveness
React quickly to user inputs.

Throughput
Complete as much work as possible.

Priority
Important tasks can get preference.

5. WHY IT MATTERS

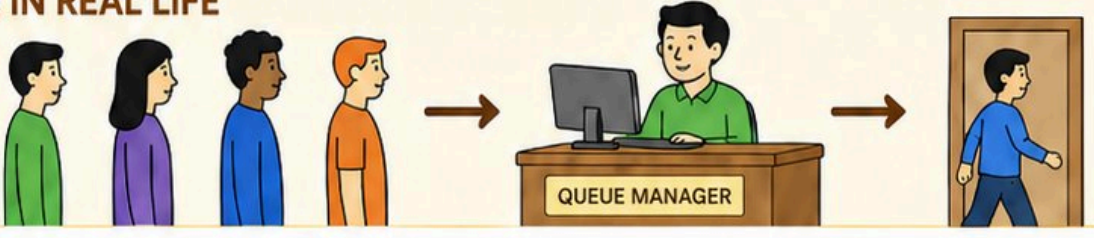
Smooth multitasking
Many apps appear to run at once.

Fast interaction
The system feels quick and responsive.

Better CPU use
Keeps CPUs busy doing useful work.

6. THE SCHEDULER IN REAL LIFE

The scheduler is like a fair queue manager at a busy service desk.



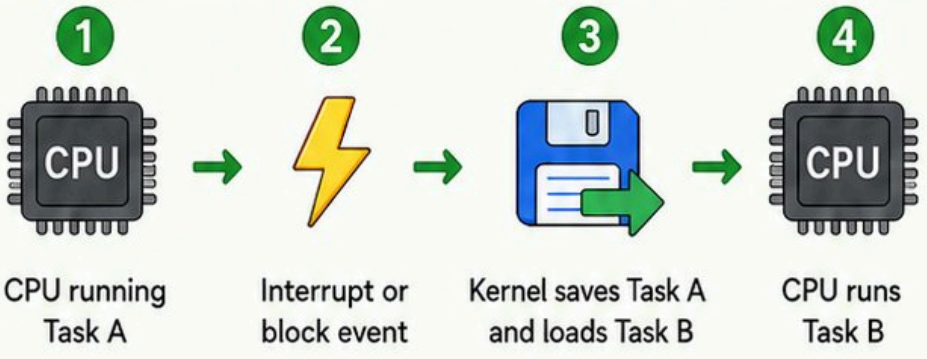
Keep it simple: Scheduling creates the feeling that many apps run at once.

Multitasking, Context Switching, and Waiting

How Linux keeps the CPU busy without wasting work

When one task pauses or its time slice ends, the kernel can save its state and run another task. This is called a context switch.

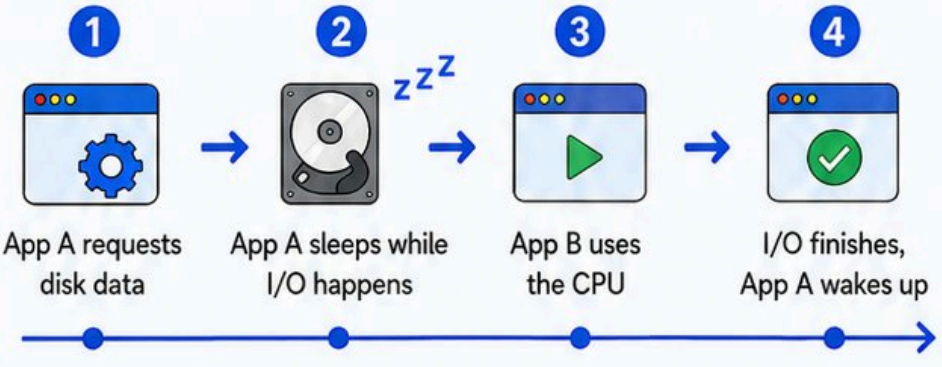
Context switch



What state gets saved?

- **Registers** — General-purpose register values
- **Instruction pointer** — The address of the next instruction
- **Stack** — Function calls, local variables, and more
- **Scheduling info** — Priority, time slice left, and other bookkeeping

Blocking and waiting



Why this is useful

- Reduces wasted CPU time
- Improves multitasking
- Keeps the system responsive

Like putting one caller on hold while the operator helps another caller.

= Task A
= Linux kernel (scheduler)
= Task B
= Waiting on I/O or time slice

Keep it simple: Linux runs something else when a task is waiting for input or I/O.

Memory Management

How the kernel gives programs memory and keeps them isolated.

Programs need memory for code, data, and temporary work. The kernel allocates memory, protects it, and reclaims it when needed.

1. VIRTUAL MEMORY

Process A
Private address space
- Code
- Data
- Heap
- Stack

Process B
Private address space
- Code
- Data
- Heap
- Stack

Each process feels like it has its own memory space.

2. ADDRESS TRANSLATION

Virtual address → Page tables → Physical RAM

The MMU uses page tables to translate virtual addresses to physical memory locations.

3. PAGES

Linux manages memory in **fixed-size** chunks called **pages.**

Physical RAM

All pages are the same size (e.g., 4 KiB).

4. PAGE FAULTS AND DEMAND PAGING

Program touches page
The page isn't in RAM.

→ **Page fault occurs**
CPU traps to the kernel.

→ **Kernel loads or allocates page**
From disk (or zero-fill) into RAM.

→ **Program continues**
The faulting instruction is retried.

Pages are only brought into memory when they are actually used.

5. SWAP AND PAGE CACHE

Swap: move out when memory is tight

RAM → Swap space (on disk) → RAM

Less-used pages can be swapped out to disk so RAM can be used for active work.

Page cache: keep disk data in RAM

Disk → Page cache (in RAM) → Page reads (data served fast)

Recently read files are cached in RAM for faster future reads.

THE BIG PICTURE: THE KERNEL MAKES MEMORY WORK

Allocation
Gives programs the memory they need, when they need it.

Isolation
Keeps each process separate and secure from others.

Reclamation
Frees and reuses memory that is no longer needed.

Caching
Keeps useful data close for speed and efficiency.

Devices, Drivers, and Interrupts

How the kernel talks to hardware.

Hardware devices all behave differently.
The kernel uses device drivers to control them through a common, safe interface.

Application
e.g., file copy

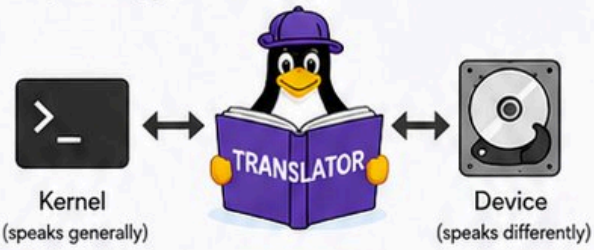
Kernel I/O layer
Provides a common, safe interface

Device driver
Knows how to talk to the device

Hardware
e.g., disk, keyboard, network card, GPU

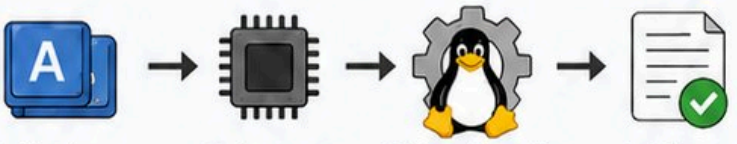
What is a device driver?

A driver is kernel code that knows how to operate a specific type of hardware.



Kernel (speaks generally) — Device (speaks differently)

Interrupts



1. **Hardware event happens** (e.g., key press, packet arrives, disk done)
2. **Interrupt sent to CPU** — Device raises an interrupt signal.
3. **Kernel handler runs** — Kernel runs the interrupt handler to handle it.
4. **Relevant work continues** — Kernel resumes normal work after handling it.

Polling vs interrupts

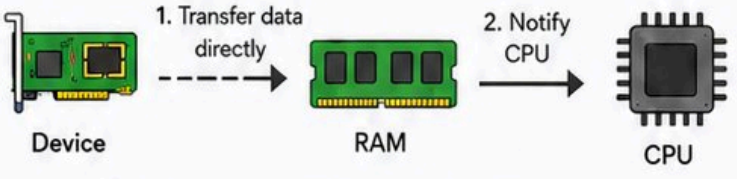
Polling (less efficient)

CPU keeps checking the device repeatedly to see if it's ready.

Interrupts (more efficient)

Device notifies the CPU only when something needs attention.

DMA (Direct Memory Access)



Direct Memory Access improves I/O efficiency.

Plug in a device

1. **Device connected** — You plug in or attach hardware.
2. **Kernel detects it** — Kernel notices the new device.
3. **Matching driver loads** — Kernel finds and loads the driver.
4. **Device becomes usable** — System can now use the device.

Examples of interrupts

- **Keyboard key press** — A key is pressed.
- **Network packet arrives** — Data arrives from the network.
- **Disk I/O completes** — Read/write operation finished.

Drivers are the kernel's translators for hardware.

Filesystems and the VFS

Why Linux makes files and devices feel consistent

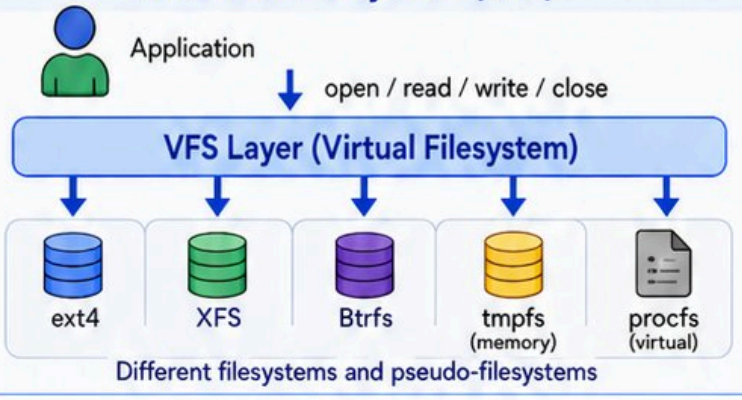
Linux often exposes resources through file-like interfaces.
This gives programs a common way to interact with many things.

1. Everything is a file?

Regular files	/dev devices	/proc process info	/sys kernel/device info
Your documents, code, images, etc.	Hardware devices like disks, USB, etc.	Information about running processes and the system.	Details about devices, drivers, and kernel state.

Not literally the same, but often accessed in a file-like way.

2. Virtual Filesystem (VFS)



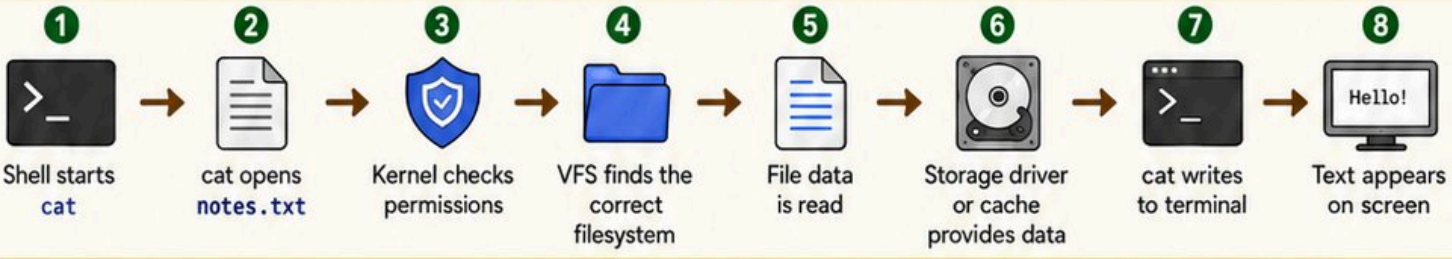
Why VFS helps

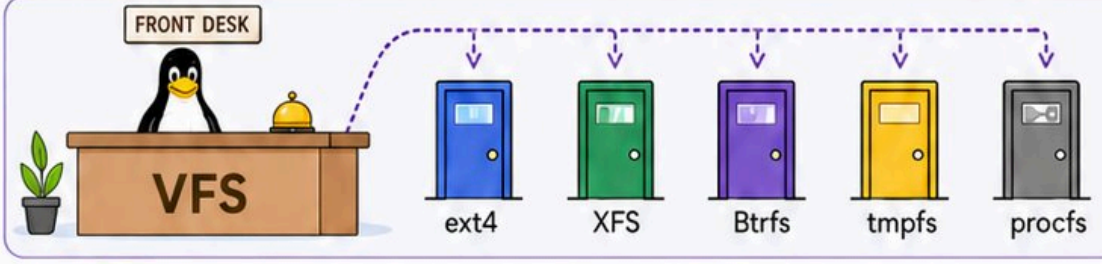
Common interface
One set of file operations (open, read, write, close, ...) works for everything.

Less hardware complexity
Applications don't need to know which filesystem or device they're using.

Easier app development
Developers focus on features, not the storage details.

3. What happens when you run `cat notes.txt`?





The VFS is like a universal front desk for many storage backends.

Keep it simple: Applications use familiar file operations, even when the backend differs.

Observe the Kernel in Real Linux

Useful commands, common myths, and a quick recap.

1. USEFUL COMMANDS

`uname -r`	running kernel version
`top / htop`	processes and CPU activity
`free -h`	memory usage
`lsblk`	block devices
`lspci`	PCI hardware
`lsusb`	USB devices
`lsmod`	loaded modules
`dmesg`	kernel messages
`/proc`	process and kernel info
`/sys`	devices and kernel objects

2. KERNEL MODULES

Some features and drivers can be loaded when needed.

3. COMMON MYTHS

MYTH: Linux is only the whole OS
FACT: More precisely, Linux is the kernel.

MYTH: High RAM use is always bad
FACT: Cache can be useful.

MYTH: Apps directly control hardware
FACT: They usually go through the kernel.

MYTH: Drivers are only for peripherals
FACT: Many subsystems use drivers.

4. QUICK RECAP

CPU: the kernel decides what runs.

Memory: the kernel allocates and protects it.

Devices: the kernel uses drivers.

Files and services: the kernel provides safe interfaces.

5. NEXT LEARNING STEPS

Processes: scheduling, states, signals

Memory: paging, swapping, caches

Filesystems: storage, permissions, mounts

Networking: sockets, protocols, netfilter

Containers: namespaces, cgroups, images

Once you understand the kernel, many Linux topics become much easier.